
X-CUBE-STL: advanced topics

Introduction

This document applies to the X-CUBE-STL Expansion Package. It includes additional explanations and guidelines on the STM32 safety concept, and related document set.

This document is written by using as reference standard the IEC 61508 international norm for functional safety of electrical / electronic / programmable electronic safety-related systems. The version used as a reference is IEC 61508-1/7:2010.

It is essential for the reader to understand and be familiar with the appropriate STM32 *MCU* safety manual and its content. This is a prerequisite to understanding this application note.





1 General information

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

arm

1.1 Normative reference

This document is written using the IEC 61508 international norm for functional safety of electrical/electronic/programmable electronic safety-related systems as reference. As a consequence, wordings and concepts inherited by such standard are not explained again in this document.

The version used as reference is IEC 61508:1-7 © IEC:2010.

ST Restricted



2 Safety manual

2.1 Assumed requirements (ASRs)

An initial safety analysis has been carried out on the general STM32 safety concept. The basic safety requirements have been derived from this analysis and allocated to specific functions and resources.

These requirements are defined as “assumed safety requirements” (ASRs) because they can be considered as directly derived from the general STM32 safety concept. The objective is to provide the *End user* with clear outcome and avoid the need for any further decisions or analysis.

Skipping the implementation of required safety mechanisms specified in the safety manual section “Hardware and software diagnostics” may have local effects on the related peripheral, to skip an ASR may completely invalidate the whole STM32 safety concept. For that reason, this part of the document provides extensive explanations on the necessity behind some crucial ASR and offers – when possible - some potential justifications for a partial or complete missing implementation.

Note: Some ASRs are “duplicated” under the form of Conditions of Use in related sections of the Safety manual (CoU_1, CoU_2, CoU_3, CoU_4 are directly related to some ASRs). This redundancy has been decided for the sake of clarity, to highlight to the End User that these ASRs have a direct impact on the use of specific STM32 modules and/or functions.

2.1.1 ASR3.1

The aim of this requirement is to provide a general framework for the computation of the base failure rate of the device. This document assumes a device is operating in a lifecycle phase between infant mortality (excluded by production tests) and the wear-out end of life. That phase is the middle zone of the well-known bathtub curve, where the failure rate can be assumed as constant. In that framework, the model selected for base failure rate, defined by IEC62380 is to be used. Note that this assumption does not provide insights on the device expected operating life under specific conditions. *End users* needing such information must refer to [Section 3.3 Reliability vs safety metrics](#).

Note: ASR3 wording used in safety manuals published before January 2023 was open to some misunderstanding on the actual aim and scope of the ASR itself. Nevertheless, the intended meaning was the same as the current, updated version.

The second statement included in this assumption is “no proof test is intrinsically needed”. It is worth reminding the reader that the scope of proof tests is to provide adequate detection for the fraction of failures which the existing on-line, automatic tests were unable to mitigate (undetected failures); usually the target for proof tests is to achieve values near to 100% of coverage. Different approaches are possible in practical implementation – in some cases, where on-line test efficiency is poor, it is possible to leverage on more effective detection provided by proof testing. In those cases, the proof testing is executed periodically during maintenance phases. The STM32 safety concept assumes that a majority of failures are directly mitigated/detected by on-line and periodical tests; indeed, more than 90% DC value can be achieved at MCU level and also on a large part of the underlying individual modules. Because of this good performance, the need for dedicated proof testing is therefore excluded. The implementation of proof testing is still possible, to target the detection of specific failures affecting modules/ function needing to execute their diagnostic during off-line periods (such as internal/external watchdogs, the clock security system, the timer break function, and so on). In such a case, the proof test provides additional safety margin to the safety concept.

2.1.2 ASR6

Behind this ASR there is the reasonable assumption that software applications implementing non-safety-related function are usually not developed according to the formal V-model of IEC 61508-3. In such a case, the coexistence of non-safety-related software with the safety applications must be achieved by additional measures. It is worth reminding the reader that software which is not developed according to a formal flow is more exposed to systematic failures, for instance, software bugs or incomplete/contradictory specifications. Such failures may lead the non-safety software interfering with the safety one, for instance overwriting wrong memory locations, wasting time deterministic functions by excessive resource occupation, and so on.



The Memory Protection Unit (*MPU*) available on the STM32 is, in principle, able to provide a correct isolation between different software tasks, assuming that a software function can correctly configure the *MPU* before giving the control to one of the possible coexisting software instances. Therefore, a combination of *MPU* dynamic reprogramming and a correct privilege levels management policy is required. In this case, both aspects must be provided by a separated software function (different from safety application and the non-safety-related software) designed for that purpose and developed with a Systematic Capability (*SC*) at least equal to the one claimed for the safety application. So, the isolation feature cannot be considered as an intrinsic STM32 capability as it strongly depends on application software implementations. In consequence, this assumed safety requirement excludes this coexistence.

End users intending to implement non-safety-related software in coexistence with the implemented safety function(s) must take the responsibility for the correct implementation and verification of an isolation concept. This, in principle, is possible, as explained above. The use of a certified safe *RTOS* offering such features is strongly recommended as the required isolation features are part of standard functions which are offered.

The implementation of a complete isolation between safety and non-safety software is not just related to memory space access – the use of shared resources (*CPU*, busses, *DMA* and so on) may lead also to a non-safety-related software to threaten the time determinism of the safety application. The use of a certified safe *RTOS* easily solves this potential issue.

2.1.3

ASR7

This assumed requirement completely excludes the implementation of safety function(s) which depends on the *MCU* correct wake-up from a low power state. There are two main reasons for such an exclusion at high level of safety analysis:

1. Wake up hardware features are poorly testable during operating time.
2. During the sleeping period the system is exposed to error accumulation, as the periodic software-based diagnostics cannot be executed. This breaks the main assumption on the prevalence of single failures mitigated in efficient way by the STM32 software-based safety concept.

For *End users* who still intended to deal with the low power states of the *MCU* may resolve the issue at architecture level, guaranteeing in any case the safety function execution or at least a deterministic transition to a safe state. This is done by using external resources outside STM32 *MCU* or by some specific assumptions at system level such as including human intervention in the safety loop assumptions.

2.1.4

ASR8

As explained by a note in this ASR, an important fraction of *CPU* failures makes it impossible to run the software correctly – such as failures causing a program counter to freeze, impossibility of fetching correct data or instructions, and so on. STMicroelectronics cannot disclose the exact fractions of the failures as they concern highly confidential intellectual property of multiple stakeholders. In any case, due to such an important effect, a safety concept which relies just on software execution to switch to safe state in case of failure would be highly questionable during the final assessment phase. In consequence, additional entities such as an external watchdog must be included in the safety architecture, as recommended in the safety manual.

The safe state transition must be correctly analyzed and documented from both the hardware and the software point of view. In the case of the watchdog intervention, end users must correctly understand the behavior of the STM32 in case a reset signal is triggered: in this case, the reset signal is internally propagated to the *CPU* and to the other modules, which are brought to reset condition. Refer to the appropriate functional documentation for related details. If software can no longer be executed, this local reset state for peripherals is the only effect of the reset signal issued by the watchdog – end users must therefore determine if that would be sufficient to guarantee the safe state switch. Depending on the safe state defined in each case, additional solutions can then be evaluated:

- Dedicated hardware solutions compatible with the local reset condition for output peripherals⁽¹⁾.
- Adoption of an intelligent watchdog capable of issuing a reset and also to generate a combination of signals which is then used to drive the system in safe state.
- Additional assumptions at system level (equivalent to a new definition of the local *MCU* safe state)⁽²⁾.

1. As an example, *GPIO* lines are brought to their “default/reset configuration” by the reset signal, so their state is expected to change accordingly. The adopted topology of the externally connected components may be able to drive a deterministic behavior, like a relay opening, and so on.
2. For instance, if the STM32 implemented safety function is to produce and send a message to an external counterpart, a reset completely prevents any messages being sent. In such a case, the external counterpart switches to safe state when no valid messages arrive within a certain time window, the STM32 reset can be considered in a valid local safe state.



Note: *Immediate switching to safe state by software execution in case of a fault detection is a recommended approach, because:*

- *it provides undisputable speed for the safe state transition*
- *it is the privileged way to manage failures which could be difficult to link to a mere watchdog timeout violation (for example, data corruption, data communication errors, and so on).*

2.2 Additional notes to safety requirements (safety mechanism/Conditions of Use)

This section collects additional information on safety mechanisms proposed by the *MCU* safety manual.

2.2.1 Information redundancy techniques on messages including end to end protection

This is applicable to the following tests:

- UART_SM_2, UART_SM_3
- SPI_SM_2, SPI_SM_4
- IIC_SM_2, IIC_SM_4
- USB_SM_2, USB_SM_3
- CAN_SM_2

These methods (as the other prescribed for the communication peripherals) are introduced to allow the detection of hardware failures affecting the components of the peripheral itself. Therefore, they cannot be considered a complete implementation of a "white channel", or "black channel" as defined by IEC 61508-2, 7.4.11.2. Neither the communication channels nor the remote counterpart parts are included in the analysis and computations. Therefore, the failure rate of the channel is not considered in the STM32 *FMEDA* and so is not included in the peripheral failure rate value reported in *FMEDA* documents.

The meaning of the generic assumption in the safety mechanism description:

"It is assumed that the remote counterpart has an equivalent capability of performing the checks described".

This is to clarify that the efficiency of the checks also depends on counterpart actions. To provide an example, data integrity of a message that is sent clearly depends on the positive checks done by the receiving counterpart, before consuming the data as valid – so, the mitigation of hardware failures affecting the STM32 peripheral assumes such checks are carried out.

Note: *In case a counterpart is unable to perform part of the operations described above (because of intrinsic limitations), the solution must be defined by the system architecture schemes. For instance, the inability to add a checksum on data frames could be solved by leveraging on multiple data transmission, or by using multiple remote input counterparts (possibly including diversity as well).*

2.2.2 Information redundancy techniques on messages

This is applicable to the following tests:

- UART_SM_2
- SPI_SM_2
- IIC_SM_2
- USB_SM_2

This class of safety mechanism is designed for communication peripherals. It is described in the safety manual as completely overlapped by the "Information redundancy techniques on messages, including end-to-end protection". For example, UART_SM_3 completely overlaps UART_SM_2. Using this mechanism in the *FMEA* gives more granularity and details to the analysis. Indeed, multiple failure modes by several peripherals are efficiently mitigated by this simpler safety mechanism, without the need to leverage on its more complex counterpart which includes end to end protection. End to end protection, such as UART_SM_3, is usually needed to address a specific list of communication failures. This is a requirement of IEC 61508-2, 7.4.11.1. but some of them could be absent from the actual systems implementations. An example is "masquerading" which cannot happen if only one entity is connected to the communication bus. Other situations are also possible.



2.2.3 CoU_4

This requirement may seem redundant. Its objective is to state the real nature of the STM32 safety concept. In fact, several safety mechanisms are more effective on a wider scope, so related evaluations must be done at *MCU* level. This safety requirement requests the end user to provide the whole summary of their decisions for the implementation structure of the listed safety mechanisms. This CoU is also used in FMEA documents to simplify the cases where a failure mitigation is provided by a multiplicity of different safety mechanism, all belonging to the highly recommended set.

2.2.4 Parity bit on SRAM

The content of this section applies to RAM_SM_1.

2.2.4.1 Considerations on SM compliance to IEC61508

Parity bit safety mechanisms is referred to in IEC 61508-2, Table A.6 – Variable memory ranges. STM32 *MCU* implementation is compliant to such description. Table A.6 indicates as maximum achievable *DC* as “Low”, which is equivalent to 60%. This must be considered as a conservative value for transient failures (soft errors) because the current safety standard does not distinguish between permanent and transient failures. Indeed, a *RAM* transient failure is composed predominantly by *RAM* cells bit flips – now, parity bit protection has a 50% *DC* for all odd bit flip multiples in the same word (which is one of the multiple failures composing the whole *RAM* set). It does, however, have a much higher *DC* capability in case of single bit flips in one word, or in separate words. So, *DC* = 60% should be considered the minimum *DC* achievable on the failure mode “RAM/information change due to soft errors” for parity bit RAM_SM_1.

2.2.5 Information redundancy for safety-related variables in application software

The content of this section applies to RAM_SM_3.

2.2.5.1 Considerations on ranking

This method is ranked as ++ (highly recommended) in STM32 devices even though a powerful *ECC* scheme is available for SRAM. It is for the following reasons:

- In some STM32 series *MCU*, no runtime test operation is available for the *ECC*. Because of this, it may not be appropriate to rely exclusively on *ECC* protection for some failures. This method proposes to add a redundant protection layer for failures related to data corruption. More details can be found in Section 2.2.6 Notes on testing methods for *ECC* schemes on SRAM, flash memory.
- SRAM data are manipulated inside the *CPU* and so they are exposed to corruption due to soft error impact on the *CPU* itself. The method helps mitigate such failures.

2.2.5.2 Considerations on SM compliance to IEC61508

This method, if implemented according to the description of the related section of the STM32 safety manual, can be considered as a combination of two different methods recognized in IEC 61508-2, Table A.6 Table A.6 – Variable memory ranges:

1. When double storage is used: A.5.7 Double *RAM* with hardware or software comparison and read/write test.
2. When encoding checksum is used: A.5.6 *RAM* monitoring with a modified Hamming code, or detection of data failures with error detection - correction codes (*EDC*).
In this case, the safety standard description can be associated to this method because in principle the same operation is done, but by software instead of automatic hardware checksum addition. The principle is the same.

The two recognized methods defined by Table A.6 promote the *DC* result from Medium to High

Note:

As a reminder, the IEC 61508-2 guidance tables combine considerations valid for both permanent and transient failures. Indeed, it can be noted that the warnings expressed in the safety standard, Table A.6, about the detection efficiency of method A.5.6 relate to permanent failures mitigation only. Because such RAM permanent failures are already addressed by another safety mechanism (RAM_SM_0), then no safety gaps are possible.

The achievable *DC* on RAM cells failures by the implementation of RAM_SM_3 must be considered as having a value somewhere between Medium (90%) and High (99%)



2.2.6

Notes on testing methods for ECC schemes on SRAM, flash memory

In some STM32 MCU series, a limitation applies to the ECC protection feature implemented for SRAM and flash memories – no hardware-based test procedure is available for the ECC mechanism. This limitation potentially exposes the STM32 safety concept to multiple failures. This is explicitly reported in RAM_SM_7 / FLASH_SM_7 safety mechanisms descriptions in safety manuals. In consequence, if a permanent failure disables any of the ECC correction or ECC detection features, further failures impacting the protected memory are no longer mitigated. From a conceptual point of view, note that the current IEC 61508 safety standard does not explicitly require the on-line test for hardware diagnostic, nor links the claimable safety mechanism diagnostic coverage to such a test. In any case, because diagnostic testing is becoming a common practice in safety (see as example ISO 26262 requirements about latent failures mitigation), this particular ECC topic must be analyzed in detail. Two different approaches are possible, a qualitative one, and a quantitative one.

Qualitative argument

The mitigation against ECC failure is based on the so-called “Swiss cheese model” – the mitigation concept is built by a sequence of independent, non-overlapped “layers of protection”, in such a way that a failure escaping to a layer is intercepted by at least one of the others. In this case, the local safety concepts for SRAM and flash memory includes additional protection mechanisms, such as, for instance, RAM_SM_1 Information redundancy for safety-related variables in application software. These safety mechanisms are already ranked ++ (highly recommended) because of the mitigation of different failures and therefore assumed to be implemented in the STM32 safety concept. These mitigation measures for ECC failures are indicated in the “Multiple-fault protection” field for RAM_SM_7 / FLASH_SM_7 safety mechanisms descriptions in the safety manuals. Additional mitigation measures can be added by leveraging on the “recommended” (+) RAM_SM_0, FLASH_SM_0 software-based diagnostic checks. These tests, which are usually available inside the X-CUBE-STL library features, are time-consuming and so their periodical execution raises the problem of correctly dimensioning the execution interval (see next subsection)

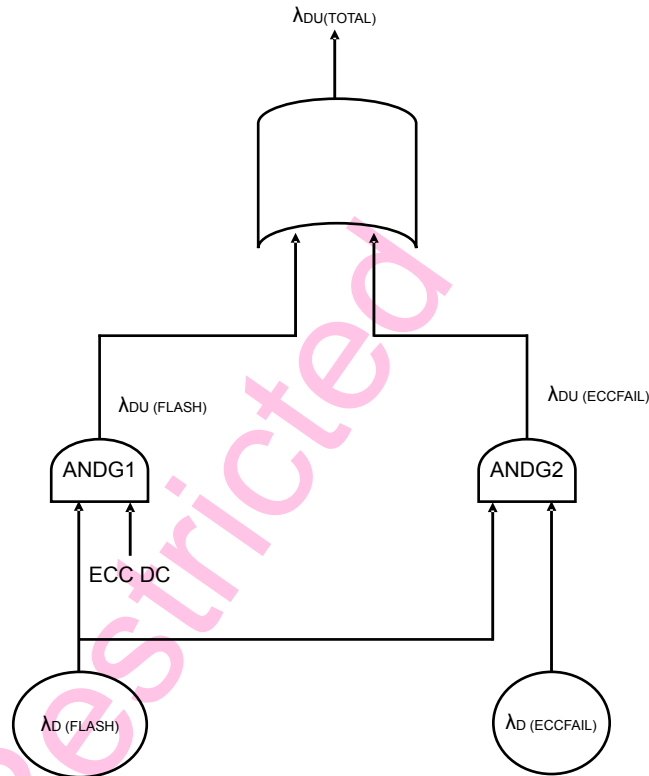
Quantitative argument

The multiple faults scenario related to the violation of safety described above need the combination of two different events:

- FLASH: A permanent failure disabling the flash memory ECC correction and detection, followed by one or multiple permanent failures in flash memory cell(s).
- SRAM: A permanent failure disabling the SRAM ECC correction and detection, followed by one or multiple permanent failures, or one or multiple bit flips in the SRAM cell(s).

For both memory types, the two events can be considered non-correlated and independent as they belong to two different, separated hardware modules, and built with different technologies (digital logics vs SRAM or NVM). The weight of common causes originating the multiple faults scenario can be therefore estimated to be negligible.

Resulting safety analysis can be represented by a dedicated FTA, the exercise is run on the above diagram on the flash memory use case:

Figure 1. Dedicated FTA flash use case


DT70136V1

$\lambda_{DU}(\text{FLASH})$ is the failure rate of the Flash memory cells.

$\lambda_{DU}(\text{ECCFAIL})$ is the failure rate of ECC logic failures leading to ECC disable.

The ANDG1 gate represent the mitigation done by the ECC on the flash memory failure rate:

$$\lambda_{DU}(\text{FLASH}) = \lambda_D(\text{FLASH}) (1 - \text{DC}_{\text{ECC}})$$

The ANDG2 gate represent the combination of the two events, failure on the flash memory cells and failure of the ECC logic. The problem is how to compute the resulting failure rate. As the two events can be assumed to be independent (see above), resulting probability can be assumed to be equal to their product. As λ are failure rates, so basically a probability divided by a time, to correctly compute the product, a time factor is needed. As a conservative approach, it is possible to assume as time factor a value representing the whole life of the device (T_{life}), for example 20 years (notional example). The related final formula is the following one:

$$\lambda_{DU}(\text{ECCFAIL}) \approx T_{\text{life}} \lambda_{DU}(\text{FLASH}) \lambda_D(\text{ECCFAIL})$$

Note:

With regards to the failure rates, a similar computation is stated in the examples provided in ISO26262:2011, part 10 section B.4 “Probability analysis using the fault tree”.

As represented by the upper OR gate, this additional unmitigated failure rate must be added in principle to the $\lambda_{DU}(\text{FLASH})$; the whole effect is basically a reduction of the diagnostic coverage achievable by the flash ECC. Computing the values for $\lambda_{DU}(\text{FLASH})$ and $\lambda_{DU}(\text{ECCFAIL})$ (see above formulas), under the assumption of associating a conservative value of 99% for diagnostic coverage achievable by the flash ECC, usually shows that $\lambda_{DU}(\text{ECCFAIL})$ is at least 3 orders of magnitude lower than $\lambda_{DU}(\text{FLASH})$; the safety margin is therefore relevant. To further lower this residual risk, it is possible to impose the execution of a periodic test by software on the flash memory cell contents. This can be for example by related safety functions offered by the X-CUBE-STL library. In such a case, each time the test is executed it is guaranteed that flash memory content is free from errors. Keeping the same approach as the above formula, it is then possible to replace T_{life} with the X-CUBE-STL test execution period; to provide an example, imposing a monthly test execution (the “reference” proof test period as proposed in [Section 2.3.3 Considerations on safety metrics targets vs the architecture](#) to lower failure rates in 1oo2 schemes). The resulting computation is further lowered by an additional two orders of magnitude.

2.3 Architecture-related topics

This section presents the topics relevant at architecture level for *End user* applications using STM32 MCUs and MPUs.



2.3.1 Notes on channel diversity

When multiple channels architecture is adopted, such as 1oo2, the “diversity” between channel arguments may play a role in the overall evaluation of the safety system. IEC 61508-4 defines diversity as “different means of performing a required function”. Diversity between channels has therefore nothing to do with the mitigation of common causes (as sometimes mistaken) which are indeed related to functions shared between channels, whose failures may affect several of them. More details on common causes of failures can be found in the related section of the *MCU* safety manual.

Diversity, instead, is related to the improvement of the systematic capability of the system. Diverse channels are less likely to be affected by the same design errors, as the functions are implemented in redundant ways by using different means and principles. This is recognized by IEC 61508-2 section 7.4.3 which allows the declaration of a higher systematic capability (*SC*) + 1 for multiple parallel channel implementation but constrained to “sufficient independence in the design”. Diversity can be applied to hardware and/or software implementation/design.

Unfortunately, IEC 61508 provides poor guidance on how to evaluate the diversity between two designs based on microcontrollers/microprocessors, whether it is in a qualitative or a quantitative way – and this brings the diversity concept into question during an assessment. Looking to this particular framework, no detailed guidance on diversity for internal functions of an *MCU* is available.

Software diversity is not discussed in this document, as this topic is under the full responsibility of the final user.

Note: The diversity concept is widely used in Tools Confidence Analysis.

Relating to hardware diversity, the following table provides some guidance on the achievable diversity by using two different *MCUs*.

Note: The term “different” means at least two different part numbers of the same STM32 series.

Table 1. STM32 series diversity examples

SoC Resource	Different p/n of same STM32 series	Different STM32 series with the same CPU	Different STM32 series with different CPUs	MCU from a different silicon vendor
CPU	No diversity	There could be some diversity in CPU components which depends on SoC integration factors (e.g. the NVIC, which synthesis depends on the number of available interrupt lines). In any case, for the inner core of the CPU “no diversity” should be assumed ⁽¹⁾ .	Refer to Arm® docs ⁽²⁾	Full diversity if no Arm® CPU is used, otherwise refer to Arm docs ⁽²⁾
MCU system architecture	No diversity as MCU system architecture is the same	Different STM32 series MCUs may use different combinations of internal connection technology (AHB, APB, bus matrix, etc.), with different peripherals aggregations on different memory space addresses. So, the End user may judge which level of diversity is achieved.		Diversity is achieved
Memories	No diversity must be assumed ⁽¹⁾			Usually, diversity can be assumed.
Peripherals				Diversity is achieved as usually the implementation of the peripherals is different

1. For internal reasons related to intellectual property protection, STMicroelectronics does not plan to provide specific information on potential differences between design instances of the same functionality or module among different STM32 MCU/MPU series devices and/or part numbers. Accordingly, the end user cannot make any assumptions/consideration on IP differences, except from what is currently reported in the STMicroelectronics publicly available technical documents (Reference Manuals and Datasheets).

2. Refer to Arm® TRM (Technical Reference Manuals), publicly available.

Lack of claimable diversity for internal resources in the *MCU* can be partially dealt with by some design solutions:

- Use of diverse communication peripherals on the two channels to implement the same input function. This can be illustrated using the *UART* on one channel and *SPI* on another, and this is of course, strongly depend on the communication capability of the external entities.



- If different part numbers from the same STM32 series are used, by adopting different instances of the same peripheral type, such as UART1 on one channel and UART3 on the other as a notional example – possibly belonging to different peripherals subsystems. This solution provides at least diversity in the use of internal SoC connections.
- Use of different *RAM* addresses: as memory access is based on addresses and data, change the addresses as data cannot be modified. For instance, to declare different base address for stack and heap in the two channels. ⁽¹⁾
- Use of linker table mapping to allocate software functions to different memory spaces in the flash memory; this way, the execution of the same instructions is done from different addresses in memory⁽¹⁾

1. *These two solutions do not provide software diversity, but they just change the way the software uses underlying hardware.*

2.3.2

SIL1 Use Case

STM32 safety concept, which is specified in the *MCU* safety manual, targets the implementation of safety functions up to SIL2 on a single microcontroller. If the safety integrity target is SIL1, the standard SIL2 approach could result in an implementation overhead, as several safety requirements can be in principle relaxed. The following set of requirements (and related considerations) provides guidance for a dedicated optimization of the safety concept.

To correctly document the SIL1 Use Case, the end user must carry out the following steps:

- To integrate *MCU* safety manual requirements with the additional requirements specified in this application note.
- To recompute the *MCU* FMEDA numbers with the updated values for achievable *DC* as per the relaxed requirements. Generic guidance on the re-computation procedure is provided in the document *How to customize the results of FMEDA on STM32 MCUs and MPUs* (AN5321).

The SIL1 Use Case proposal is based on following assumptions (ASx):

- AS1: The SIL1 Use Case includes waivers to the requirements included in the STM32 *MCU* safety manual, to allow the use of the *MCU* for the implementation of safety functions up to SIL1 according IEC 61508.
- AS2: The SIL1 Use Case set assumes the range 60%<-->90 % as formal target SFF for the overall *MCU*, according IEC 61508 2 Table 3 (for HFT=0)
- AS3: The SIL1 Use Case set assumes the range of 60% to 90% as reference target SFF for failure modes computed by considering separately each individual technology available in the STM32 *MCU*: digital logic, non-volatile memory, volatile memory, analogue, package⁽¹⁾ (1).
- AS4: The SIL1 Use Case assumes that there are no SFF or *DC* local targets for STM32 *MCU* individual modules.⁽²⁾
- AS5: The SIL1 Use Case assumes the following items as failure mitigation strategy:
 - AS5a: There are no *MCU* modules with all failure modes unmitigated.
 - AS5b: A certain number of unmitigated failure modes is allowed for any *MCU* module⁽³⁾
 - AS5c: Current SIL2 requirements coming from dependent failure analysis are confirmed to be part of the SIL1 Use Case
 - AS5d: Package reference *DC* cannot be the same assumed for SIL2⁽⁴⁾

1. While AS2 is a direct consequence of IEC 61508 2 requirements according to the *MCU* safety manual definition for “compliant item”, AS3 has been placed to avoid relying on SFF target achievement at *MCU* level (which can be down to 60%) just by leveraging on technologies with higher permanent base failure rate (non-volatile and volatile memory) when computed by using IEC62380.
2. As per the AS4 consequence, SIL1 Use Case could obtain local *DC* values on individual module below the threshold of 60%. This effect is mitigated by the combination of AS5a and AS5c.
3. AS5b basically allows a limited number of failure modes to be left completely unmitigated for some *MCU* modules. Despite there being no mandatory and explicit IEC 61508 requirements blocking this assumption, it is highly recommended to discuss and validate with the final assessor the acceptance of the assumption. It is worth noting that the combination of AS3 and AS5a decreases the potential criticality of AS5b assumption.
4. The standard *DC* value used in FMEDA computations for “package” in SIL2 corner cases (99%) is no longer realistic for the SIL1 Use Case. While package *DC* and SFF will certainly comply with the minimum value required in the interval fixed by AS3, the end user is strongly encouraged to execute its own pin-level package FMEDA to calculate and use a more realistic value.

As described above, the SIL1 Use Case differs from the “standard” SIL2 one by a different ranking of the safety mechanisms and requirements. Resulting modifications to the summary table “List of safety requirements” included in section 3.7 of *MCU* safety manual are as follow:



- REQ1: Implementation of any safety mechanisms rated as “+” or “o” can be disregarded.
- REQ2a: All safety mechanisms rated as “++” must be implemented in the SIL1 Use Case, except the ones excluded by a dedicated requirement of this list.
- REQ2b: The implementation of safety mechanisms must follow the implementation guidelines reported in the safety manual, except if rectified by a dedicated requirement in the following item.
- REQ3: FLASH_SM_0 can be implemented by using a simple signature method using IEC 61508:7 A.4.2 Modified checksum.
- REQ4: RAM_SM_0, RAM_SM_2, and RAM_SM_3 can be replaced by hardware-based protection: RAM_SM_1 parity bit or RAM_SM_7 ECC protection (if available in the selected STM32 device). In case RAM_SM_1 is used, the DC value for the SRAM must be reduced to 60% for both fault models. In case RAM_SM_7 is used, the DC value for the SRAM must be reduced to 90% for both fault models. This is a conservative value.

Important: *If applicable to the adopted STM32 series, the issue related to missing test mechanisms for the selected hardware-based diagnostic should be discussed with the assessor. Refer to [Section 2.2.6 Notes on testing methods for ECC schemes on SRAM, flash memory](#) for details.*

- REQ5: For communication peripherals, implementation of “Information redundancy techniques on messages, including end-to-end protection” (for example UART_SM_3) can be replaced by “Information redundancy techniques on messages” (for example UART_SM_2). The whole DC value for the peripheral must be reduced to 60% for both fault models (conservative value).

2.3.3 Considerations on safety metrics targets vs the architecture

For final applications based on STM32 MCUs/MPUs compliant with IEC 61508, the most challenging target for absolute metrics is related to the SIL3 corner case in case of high demand or continuous mode (for low demand, metric targets are much higher and of less concern). Generally, STM32 devices are characterized by a relatively limited λ_{DU} which is well below the PFH target for SIL3.

A few issues may arise in case of STM32 devices with large SRAM banks widely used for safety-related data or programs, where the failure rates related to transient failures can reach high values; As transient failure rates increase with the altitude, that factor must also be considered.

The solution may come from a detailed analysis of the actual IEC 61508 requirements for such corner case. It is worth reminding the reader that assuming SIL3 as target, the 1oo2 architecture scheme (two microcontrollers) must be selected. The PFH computation is then performed with the guidance of IEC 61508-6, B 3.3.2.2 equations:

$$t_{CE} = \frac{\lambda_{DU}}{\lambda_D} \left(\frac{T_1}{2} + MRT \right) + \frac{\lambda_{DD}}{\lambda_D} MTTR$$

Eq. 1

$$PFH_G = 2 \left((1 - \beta_D) \lambda_{DD} + (1 - \beta) \lambda_{DU} \right) (1 - \beta) \lambda_{DU} t_{CE} + \beta \lambda_{DU}$$

Eq. 2

where:

- T_1 = time interval for proof test
- $MTTR$ = mean time to restoration
- MRT = mean repair time
- β = fraction of undetected failures that have a common cause
- β_D = fraction of detected failures that have a common cause
- DC is assumed to be around 90% as each channel must be SIL2 compatible.
- t_{CE} = channel equivalent mean down time (hour)

Relating to Eq. 1, the following assumptions and simplifications can be done:

- MRT can be assumed to be equal to $MTTR$ using reasonable value of eight hours for both MTT and MMTR.
- Dedicated test and maintenance procedures for SRAM like March test and memory scrubbing⁽¹⁾ can be assumed to be executed as part of the proof test⁽²⁾. In such a case it is reasonable to have $T_1 \gg MTTR$ for example: $T_1 = \text{month}$ ⁽³⁾.

1. When using an ECC scheme: to write back the right values to SRAM cells when a single errors correction is notified, and so to avoid error accumulation.



2. Execution of time-consuming tests like March test or scrubbing as spread over the T1 interval is not in contradiction with the proof test definition provided by IEC 61508 - so in principle that solution is allowed.
3. Assuming: $MRT = 8h$, $T1 = 1$ months means assuming T1 nearly two order of magnitudes greater than MRT

Resulting simplification leads to the following result:

$$t_{CE} \approx (1 - DC) T1 / 2$$

Relating to controversial parameters β and β_D used in Eq. 2, it is reasonable to suppose $\beta = 2\beta_D$ with β usually around few percentage points, for example 5%,

$$\text{so } (1 - \beta) \approx (1 - \beta_D) \approx 1$$

Note: The 5% example is assuming well-designed, separated channels.

Eq.2 can be therefore simplified:

$$PFH \approx \lambda_{DU} (\beta + 2 t_{CE} \lambda_D)$$

Note that in presence of the assumed values stated above for the proof test interval T1, the product ($t_{CE} \lambda_D$) is orders of magnitude lower than β , bringing to final simplification:

$$PFH \approx \beta \lambda_{DU}$$

To review β definition: fraction of undetected failures that have a common cause. λ_{DU} value as reported from FMEDA documents for STM32 devices is composed by two different contribution, permanent failures and transient failures. It is reasonable that the percentage β depends on the type of fault model, so leading to following equivalence:

$$PFH \approx \beta_{(perm)} \lambda_{DU(perm)} + \beta_{(trans)} \lambda_{DU(trans)}$$

While for permanent failure it can be reasonably expected that a certain percentage of undetected failures might have a common cause (for example temperature, supply voltage, clock alterations), for transient failures things are different. SRAM bit flips due to soft errors are caused either by alfa particles coming from radiating material in the IC package, or neutron particles coming from solar radiation – in both cases, it is very unlikely (for geometric reasons related to physical separation between the two MCUs) that the same single common cause may interfere with the SRAMs cells of the two separated channels at nearly the same time (time is key for transient failures) and with similar end effects. Consequently, it is reasonable to assume that the β value for transient failure would be very small (assumed to be below 1%) and so granting at least an additional, conservative two orders of magnitude reduction factor for the resulting PFH value. Therefore, under all of the above assumptions, even the SIL3 HD/CM corner case seems easily manageable by using the 1oo2 architecture, with the sole additional requirement of an additional SRAM proof test with time interval of around 1 month.

3

FMEA/FMEDA

This section includes clarifications of *FMEA* and *FMEDA* document content, their interpretation and related underlying assumptions.

3.1 Notes on safety analysis flow

The STM32 devices safety analysis is executed according to the guidance of IEC 61508 (with focus on requirements described in part 2 of the standard), and is composed by multiple different phases, described below with greater detail. Some parts of the analysis produce artifacts (documents) which are part of the *CPU* documentation in the X-CUBE-STL safety package.

Qualitative analysis: FMEA flow

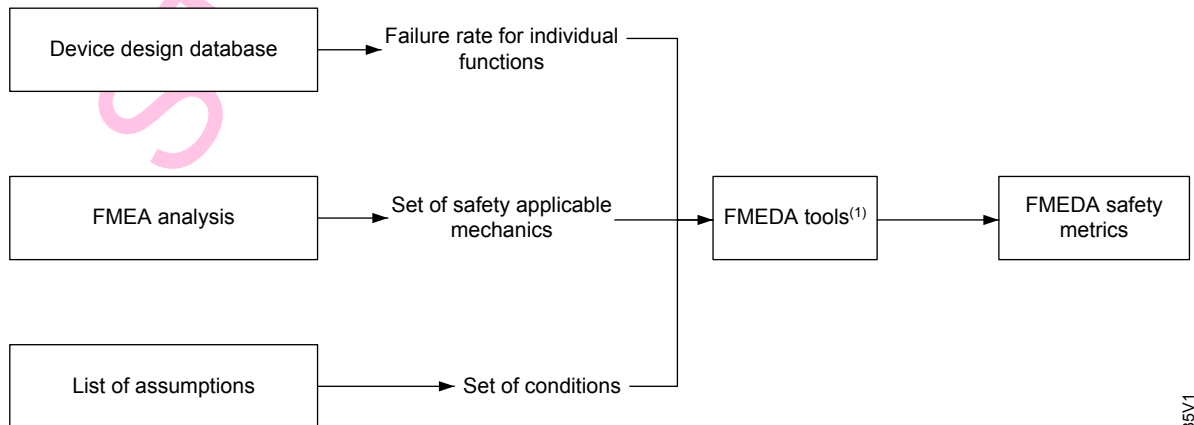
A detailed *FMEA* analysis is executed to explore all potential failure modes of the device. Each module is broken down to a set of specific hierarchy levels, and for each sub-modules the list of failures is defined. The guidance for the minimum set of failures to be analyzed is provided by tables in IEC 61508-2 Annex A. For each failure, one or multiple safety mechanisms are associated with the objectives of achieving at least a “medium” diagnostic coverage, when possible. Both Permanent and Transient fault models are analyzed, often with different combinations of proposed safety mechanisms, as their efficiency depends on the addressed fault model.

As a result, no failure mode remains unmitigated (except few cases of 100% no-effect failures). This means, there is always at least one proposed safety mechanisms or a Condition of Use detecting or mitigating the failure effects.

The outcome of the above-described analysis is available for the *CPU* in related document “Results of *FMEA* on STM32...”. The document is provided as evidence of the compliance to the analysis to IEC 61508 requirements, and to allow the *End user* to correctly understand the relevance and the role of each prescribed safety mechanism, as they are effective on specific failure modes.

Quantitative analysis: FMEDA flow

Figure 2. Quantitative analysis: FMEDA flow



Note 1: ST proprietary based on detailed design knowledge (gate counts, functions implemented at the modules) and ST safety metrics computation methodology, not fully available for the user

D170135V1

The weights of the failure modes are computed by using information coming from the real design netlist and adopting related base failure rates for permanent and soft errors, by considering the different technologies available on the device. More details on base failure rate computation can be found in [Section 3.2 Considerations on Base Failure rate](#).

Conservative elements of *FMEDA* computations:

- All *MCU* modules are considered safety related and so included in the computations.



- For *RAM* transient failures, the argument of “no-effect” failures is not used. This is a kind of reasonably conservative approach, because not considering the almost certain derating effects depending on:
 1. Software overwrites on corrupted cells before they are read, such as a stack.
 2. Data for which one single bit flip is not able to violate the safety function, such as audio/video data buffers.
 3. Unused memory cells
 4. “Temporary” use of *RAM* by the application software, such as all the C temporary variables allocated in the stack.
- No failures, nor modules are declared “safe” as per IEC 61508-4, and so they never contribute to “artificially” raise the value of the SFF. No safe argument is used because the safety concept is a general one and so the definition of the safe state is unknown (even if some statements on that can be found in the *MCU* safety manual)

DC values associated to *MCU* failure modes and to the assumed combination of safety mechanisms are decided by estimations (based on IEC 61508-2 Annex A tables/guidance, the *FMEA* outcomes and expert judgement). In case of multiple safety mechanisms overlapping on the same failure mode, the resulting *DC* is evaluated in a conservative way (so, not combining individual *DC* values as each safety mechanism is active on separated group of faults). In a similar way, the abstraction level of the failures modes aggregation for each module is decided by considering the nature of STM32 *MCU* safety concept, which is mainly software and application based.

The only exceptions are the *DC* values associated to *CPU* failure modes (permanent fault model) which come from fault injection verification activity (in the framework of the X-CUBE-STL certification) – so these last ones are actually simulated. It is worth noting that as X-CUBE-STL scope for the *CPU* is the inner core, the related coverage value is not directly visible in the *FMEDA* as the contribution is merged with many others at application level because the reported detail level is “CPU SUBSYSTEM”.

Important: *A major part of the safety mechanism defined at MCU level are application dependent – therefore, using estimated values is the most reasonable method to validate the whole MCU safety concept (because actual final application software is not available, so assumptions are needed).*

Because the STM32 *MCU* can be considered as an “element”, then IEC 61508:2010 does not mandate the achievement of specific *DC*/SFF target values for its internal modules, so down in hierarchy levels. Anyway, to avoid any criticism on the way the whole SFF target is achieved, the STM32 safety concept is built on “harmonized” modules *DC* levels, which lay around the 90% target for the large part of modules. This is true also for the different technology uses (digital logic/*RAM*/flash memory). In this way, the resulting whole achievable *DC*/SFF at *MCU* level is less questionable as its dependency on base failure rates computation is small. This can be easily verified by inspecting the *FMEDA* outcome.

Additional analysis

Other additional analyses on the *MCU* are performed (when needed):

- Event tree analyses (*ETA*) is usually adopted to derive requirements depending on potential safety issues during boot operations in complex devices.
- Fault tree analyses (*FTA*) is adopted to analyze the cross relationship between diagnostics circuitry, and to explore diagnostic disable scenarios (caused by intervening failures).

3.2 Considerations on Base Failure rate

“Base failure rate” for a given component is the failure rate associated to an individual, specific subset of the component itself: for example, portion of the silicon area, transistor, memory bits. The base failure rate is a controversial argument as different methods (accelerated tests) and mathematical models exist in the industry. The aim of this section is to describe the procedures followed in STM32 safety documentation and so to allow the *End user* to understand if an adaptation is necessary in the framework of its system safety analysis.

3.2.1 Permanent failure rate

The permanent base failure rate in *FMEDA* is computed according to the model included in IEC62380. This standard is quite old, but that model is used basically in identical way in the state-of-the-art safety standard for microelectronic devices ISO26262:2010. Base failure rate is computed adopting conservative assumptions.



As the base failure rate depends on multiple factors (the most important one is mission profile), dedicated parameters and procedures are included in *FMEDA* documents and in AN5321 to allow data customization by *End user*.

Using data from STMicroelectronics quality service (*FIT* rate from *HTOL* tests, and so on) is not recommended as this could be difficult to be reconciled with data related to other components on the *End user* board. So, the *End user* must pay attention to differences in confidence level and in component test procedures. A procedure for *FMEDA* recomputation is available in AN5321.

3.2.2 Transient failure rate

Transient base failure rate depends on multiple factors. As it is composed by two elements, alpha particles (coming from unavoidable radioactive contamination of device package) and neutron particles (solar radiation), there is a strong dependency on multiple parameters like package type (LA or ULA), altitude, possible shielding of the final application hosting the device. For this reason, a reference value for the base failure rate is used in *FMEDA* computation. The value is derived from *ITRS* tables (sea level is assumed), and it is around 1250 *FIT* / Mbit, so also in line with the predictions of the well-known SN29500-2:2010 Siemens norm (section 3, prescriptions for “sporadic failures”). The “reference” nature of the values is indicated in *FMEDA* documents; indeed, this approach allows end users to make a comparison with other competitor devices. The SER data can be also obtained from STMicroelectronics quality department upon request and used to recompute the transient failure rate. SER data also includes models to evaluate the altitude effect on the values.

Note: In such a case, the *End user* just needs to derive its new value for *FITs*/Mbits and use it together with the reference value above indicated to scale all transient failure rates (λ_d , λ_{du} , λ_s). Transient relative metrics do not change as they do not depend on base failure rate.

It is worth reminding the reader that when transient failure rate computations for devices with large *RAM* banks may produce high absolute numbers in terms of *FITs*, those numbers should be considered a pessimistic corner case, as it is not considering multiple existing factors to derate the whole failure rate. Refer to [Section 3.4 SRAM undetected transient failure rate mitigation/derating arguments](#) for more details.

3.3 Reliability vs safety metrics

Functional safety and reliability are two different aspects of dependable systems - because of that, they share multiple concepts, notations, formulas – sometimes indicating with the same wording two different things. This may lead to misunderstand documents and information, which often cannot be simply reused in the framework of one of the two domains, if generated in the scope of the other.

There are several reasons to prevent the reuse of metrics and requirements from the safety domain to that of reliability, some of them are listed here below.

In the safety domain, once a failure is detected, it is usually managed by switching the system in safe state, and so any error recovery or correction is scarcely present in the strategies or instructions proposed by the safety manual. Consequently, safety failure rates post-diagnostic (λ_{DU}) do not have much in common with reliability data, as they are computed by considering the effect of the combination of recommended diagnostics.

Safety failure rates in *FMEDA* documents are computed by using specific models (for example IEC62380) to facilitate the reuse of such data combined with the ones of other components on the board, for related *FMEDA* (refer to [Section 3.1 Notes on safety analysis flow](#)), while reliability data comes usually from stress-based test such as *HTOL*. There are therefore big differences in the methods that are used and possibly in the adopted confidence levels too, so again the “reuse” of the metrics can lead to huge mistakes.

Transient failures, also known as soft errors, are managed in a very different way for both domains. They usually do not appear in reliability data coming from *HTOL* tests nor accelerated stress tests as the aim of those tests is to reveal any permanent damage to the device. They are, however, an integral part of safety metrics, even if provided in separate way.

It is important to note, SER data are sometimes necessary in the case of a reliability framework, in particular for devices equipped with large *RAM* banks. This is done in the spirit of what is usually carried for data centers. In such a case, the best choice is for end customer to refer to the SER data from STMicroelectronics quality service (refer to [Section 3.2.2 Transient failure rate](#)).

STM32 users interested in the reliability of the device must refer to STMicroelectronics quality department information and guidance. One example is the type of document called “STM32xx Series lifetime estimates” which provides extrapolated information on device expected lifetime for a set of temperature and supply voltage profiles.



3.4

SRAM undetected transient failure rate mitigation/derating arguments

FMEDA data are application agnostic. Accordingly, the failure rate for SRAM is computed under the conservative assumption that all data in the SRAM are safety related. Resulting in a value for transient failure rate, undetected failures given by λ_{DU} value can, therefore, be relevant in the case of devices with large memory banks. To reduce this value, the *End user* can refine in its *FMEDA* that computation, leveraging on multiple possible arguments:

- The SRAM capacity is only partially used. As failures caused by soft errors (bit flips) cannot propagate to other cells, the contribution of unused SRAM locations can be declared non safety related and so be excluded from the computation.
- Previous argument can be applied to *RAM* variable related to non-safety related data.
- Some types of data stored in *RAM*, despite being safety related in principle, can have an intrinsic characteristic depending on the application, allowing their partial exclusion from safety computations. For example, in presence of display graphic raw image data refreshed with a frequency greater than the system *PST*, it is reasonable to assume that single bit flips in an image are not able to violate the safety of the system – accordingly, the graphic data failure rate can be granted by a relevant derating factor of for example 99%.

If one or more of the above arguments are applicable, the *End user* must split the whole SRAM line in multiple lines for the concerned *FMEDA*. The objective is that for each argument, the related failure rates are computed in proportion of the whole SRAM capacity. In this way, the use of the appropriate derating factors is easier and more readable during the final assessment of the system.



4 X-CUBE-STL

This section provides advanced information on the X-CUBE-STL diagnostic test library.

4.1 Notes on artificial failing function

X-CUBE-STL provides an intrinsic function, called “artificial failing”, which causes the library to produce any of the possible error messages, with minimum external instrumentation. As one of the mandatory requirements to prove the correct integration of the *STL* into the final application, the software is to exhaustively verify the correct management (including the reaction chain) of error reporting. This *STL* feature reduces the complexity of such a verification. The reduction of the required instrumentation helps to mitigate the unavoidable interference of the verification activity with the original application software.

While the verification activity described above is clearly needed, the *End user* must be aware that some corners still need to be verified elsewhere. Artificial failing mimics the situation of an error inside the *CPU*, but it cannot emulate the collateral malfunctions caused by an actual hardware failure. As explained in MCU safety manual and in [Section 2.1.4 ASR8](#) of this document, part of *CPU* failures can:

- stop any further execution of the application software,
- cause jumps of the program counter to a wrong code section,
- cause unintended read or writes access to memory.

An exhaustive verification of these additional corners with standard software simulations is often unfeasible for practical reasons:

- The emulation of the failure effects is just partial, and too intrusive when instrumented into the application software.
- Some corners also include the external watchdog,
- As the time dimension is added to the verification scenario (when the fault emerges), the achievable coverage of the verification activity is often limited.

A smarter verification strategy requires a combination of some simulation campaign together with deductive-based arguments. For instance, the proof of the efficiency of a software-based diagnostic may leverage on the systematic capability of the adopted software development model.



Revision history

Table 2. Document revision history

Date	Version	Changes
28-Jun-2023	1	Initial release.
26-Oct-2023	2	Updated the following: • Section Introduction • Section 1 General information • Section 2.1.4 ASR8 • Section 2.2.2 Information redundancy techniques on messages • Section 2.2.6 Notes on testing methods for ECC schemes on SRAM, flash memory • Section 2.3.1 Notes on channel diversity • Section 2.3.2 SIL1 Use Case • Section 3.1 Notes on safety analysis flow • Section 3.2.2 Transient failure rate • Section 3.3 Reliability vs safety metrics • Section 3.4 SRAM undetected transient failure rate mitigation/derating arguments



Glossary

ADC analog to digital converter	PFH probability of failure per hour
ASR assumed safety requirement	PST process safety time
CM continuous mode	RAM random access memory
CPU central processing unit	RTOS real time operating operating system
DC diagnostic coverage	SC system capability
ECC error correction code	SoC system on chip
EDC error detection - correction code	SPI serial peripheral interface
End user individual person or company who integrates <i>Device</i> in their application, such as an electronic control board	STL self-test library
ETA event three analyses	UART universal asynchronous receiver transmitter
FIT failure in time	
FMEA failure mode effect analysis	
FMEDA failure mode effect diagnostic analysis	
FTA fault three analyses	
FTA fault three analyses	
HD high-demand	
HTOL high-temperature operating life reliability test	
ITRS international technology roadmap for semiconductors	
MCU microcontroller unit	
MPU microprocessor unit	
MPU memory protection unit	
MRT mean repair time	
MTTR mean time to restoration	
NVM none volatile memory	
PFD probability of dangerous failure on demand	



Contents

1	General information	2
1.1	Normative reference	2
2	Safety manual	3
2.1	Assumed requirements (ASRs)	3
2.1.1	ASR3.1	3
2.1.2	ASR6	3
2.1.3	ASR7	4
2.1.4	ASR8	4
2.2	Additional notes to safety requirements (safety mechanism/Conditions of Use)	5
2.2.1	Information redundancy techniques on messages including end to end protection	5
2.2.2	Information redundancy techniques on messages	5
2.2.3	CoU_4	6
2.2.4	Parity bit on SRAM	6
2.2.5	Information redundancy for safety-related variables in application software	6
2.2.6	Notes on testing methods for ECC schemes on SRAM, flash memory	7
2.3	Architecture-related topics	8
2.3.1	Notes on channel diversity	9
2.3.2	SIL1 Use Case	10
2.3.3	Considerations on safety metrics targets vs the architecture	11
3	FMEA/FMEDA	13
3.1	Notes on safety analysis flow	13
3.2	Considerations on Base Failure rate	14
3.2.1	Permanent failure rate	14
3.2.2	Transient failure rate	15
3.3	Reliability vs safety metrics	15
3.4	SRAM undetected transient failure rate mitigation/derating arguments	16
4	X-CUBE-STL	17
4.1	Notes on artificial failing function	17
	Revision history	18



List of figures

Figure 1. Dedicated FTA flash use case 8

Figure 2. Quantitative analysis: FMEDA flow 13

ST Restricted



List of tables

Table 1. STM32 series diversity examples 9

Table 2. Document revision history 18

ST Restricted

**IMPORTANT NOTICE – READ CAREFULLY**

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice. Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgment.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2023 STMicroelectronics – All rights reserved

ST Restricted